8. Design WhatsApp/Slack Chat Messaging

Key Terms:

Term	Full Name	What it does
WebSocket	Full-duplex protocol	Persistent connection for real-time bidirectional messaging
ACK	Acknowledgment	Confirmation that message was received
Fanout	Message distribution	Sending one message to many recipients
TIMEUUID	Time-based UUID	UUID that's sortable by creation time
APNs	Apple Push Notification service	Delivers notifications to iOS devices
FCM	Firebase Cloud Messaging	Delivers notifications to Android/iOS
E2EE	End-to-End Encryption	Only sender and recipient can read messages

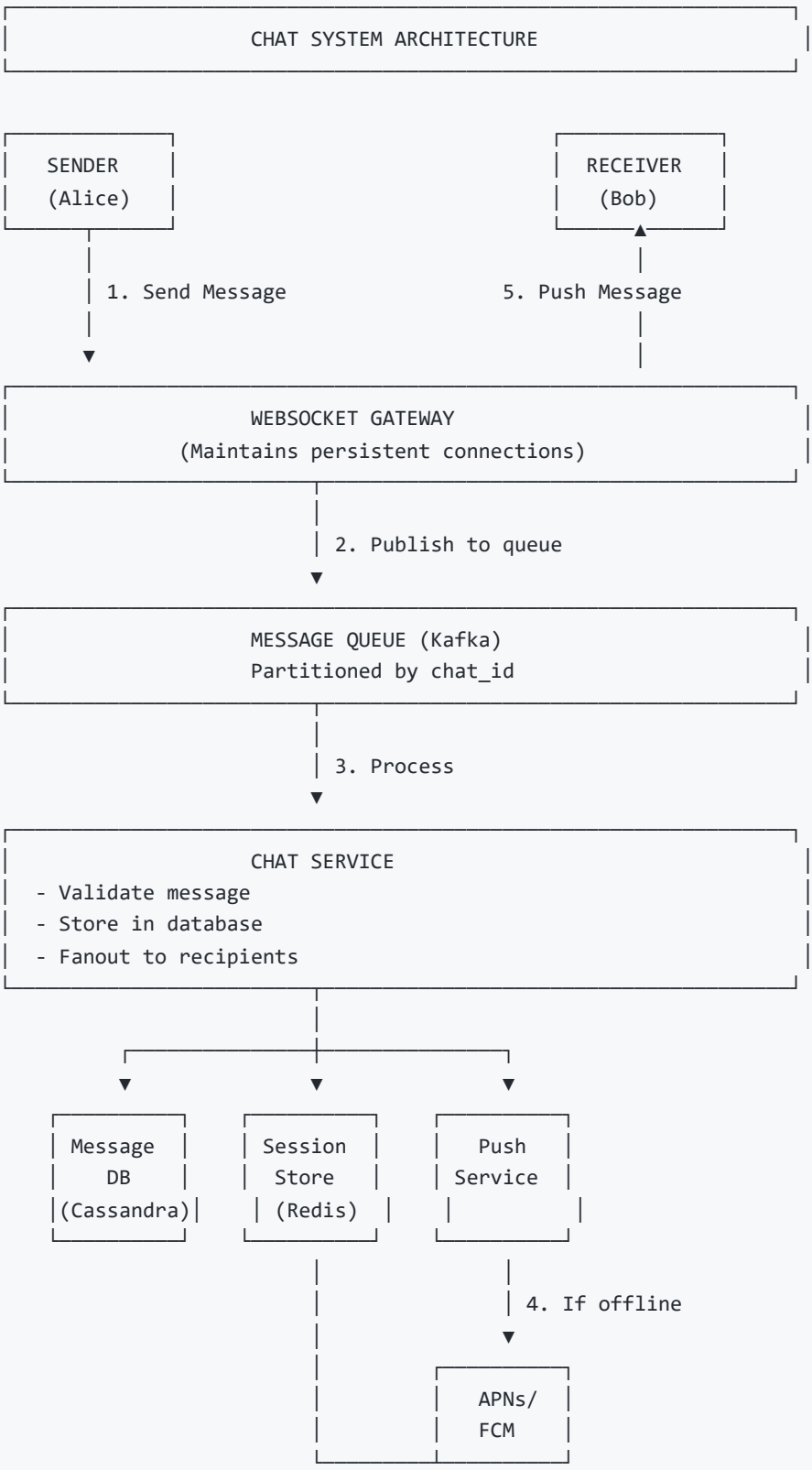
Problem Statement

Design a real-time messaging system supporting billions of messages daily with delivery guarantees, read receipts, and offline support.

Clarifying Questions to Ask

1. **Scale**: How many users? Messages per day? (2B users, 100B messages?)
2. **Message types**: Text only or media too?
3. **Group size**: Max participants per group? (256? 1000?)
4. **Delivery guarantees**: At-least-once or exactly-once?
5. **Message retention**: How long to store messages?
6. **End-to-end encryption**: Required?
7. **Features**: Read receipts, typing indicators, presence?

High-Level Design



API Design

Message APIs

Method	Endpoint	Description
`POST`	`/api/v1/messages`	Send a message
`GET`	`/api/v1/messages?chatId={id}&limit=50&before={cursor}`	Fetch message history
`PUT`	`/api/v1/messages/{messageId}`	Edit a message
`DELETE`	`/api/v1/messages/{messageId}`	Delete a message
`POST`	`/api/v1/messages/{messageId}/read`	Mark message as read

Chat/Group APIs

Method	Endpoint	Description
`POST`	`/api/v1/chats`	Create a 1:1 or group chat
`GET`	`/api/v1/chats`	List user's chats
`POST`	`/api/v1/chats/{chatId}/members`	Add member to group
`DELETE`	`/api/v1/chats/{chatId}/members/{userId}`	Remove member

WebSocket Events

Event	Direction	Payload
`message.new`	Server → Client	`{ chatId, messageId, content, sender, timestamp }`
`message.delivered`	Server → Client	`{ messageId, userId, timestamp }`
`message.read`	Server → Client	`{ messageId, userId, timestamp }`
`user.typing`	Client → Server	`{ chatId, isTyping }`
`user.online`	Server → Client	`{ userId, lastSeen }`

Send Message Request

```
POST /api/v1/messages
{
  "chat_id": "chat_123",
  "content": "Hello!",
  "type": "text",
  "reply_to": "msg_456",       // optional
  "client_msg_id": "uuid-789"  // for idempotency
}
```

Key Components

Component	Purpose	Technology
WebSocket Gateway	Persistent connections	Custom / SignalR
Session Store	Track user connections	Redis
Message Queue	Decouple and order	Kafka
Chat Service	Business logic	Microservice
Message Store	Persist messages	Cassandra
Push Service	Offline notifications	APNs, FCM
Media Service	Handle attachments	Blob Storage + CDN

Message Flow

```
1:1 CHAT:
  Alice → Server → Bob (direct delivery if online)
                 → Store for later (if offline)

GROUP CHAT (Fanout-on-write):
  Alice sends to Group(Alice, Bob, Charlie)
  Server writes to:
    - Bob's inbox
    - Charlie's inbox
  Pros: Fast reads
  Cons: Storage cost, write amplification

GROUP CHAT (Fanout-on-read):
  Alice sends to Group
  Server stores once in Group's messages
  Readers pull from Group
  Pros: Less storage
  Cons: Slower reads, complex queries
```

Message Delivery States

```
┌─────────────────────────────────────────────────────┐
│  SENT (✓)      → Server received the message        │
│  DELIVERED (✓✓) → Recipient's device received       │
│  READ (✓✓ blue) → Recipient opened the chat         │
└─────────────────────────────────────────────────────┘

Implementation:
  1. Client sends message → Server ACKs → Show single tick
  2. Server delivers to recipient → Recipient ACKs → Double tick
  3. Recipient opens chat → Send read receipt → Blue ticks
```

Database Schema

```
  -- Messages (Cassandra - partitioned by chat_id)
  CREATE TABLE messages (
      chat_id UUID,
      message_id TIMEUUID,  -- Sortable by time
      sender_id UUID,
      content TEXT,
      content_type VARCHAR,  -- text, image, video
      created_at TIMESTAMP,
      PRIMARY KEY (chat_id, message_id)
  ) WITH CLUSTERING ORDER BY (message_id DESC);

  -- User Inbox (for quick access to recent chats)
  CREATE TABLE user_inbox (
      user_id UUID,
      chat_id UUID,
      last_message_id TIMEUUID,
      last_message_preview TEXT,
      unread_count INT,
      updated_at TIMESTAMP,
      PRIMARY KEY (user_id, updated_at)
  ) WITH CLUSTERING ORDER BY (updated_at DESC);
```

Scalability Considerations

- **Connection management**: Each gateway server handles ~500K connections
- **Partitioning**: Kafka partitioned by chat_id for ordering
- **Presence**: Use heartbeats + Redis pub/sub for online status
- **Group fanout**: Limit group size, use async fanout for large groups
- **Message ordering**: TIMEUUID ensures global ordering within a chat

Trade-offs

Decision	Option A	Option B	Recommendation
Protocol	HTTP polling	WebSocket	**WebSocket** - real-time, efficient
Group fanout	On write	On read	**On write** - for small groups
Message store	SQL	Cassandra	**Cassandra** - write-heavy scale
Ordering	Timestamps	Sequence IDs	**TIMEUUID** - sortable + unique

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Message ordering and delivery guarantee	Tech Guide
Redis	Online presence and connection state	Tech Guide
Cassandra	Write-optimized message storage	Tech Guide

How to Explain It Well

> **Opening**: "A chat system needs real-time delivery with offline support. I'll design for 2 billion users and 100 billion messages daily."

> **Key talking points**:
>
> - "WebSocket gateways maintain persistent connections - one per user"
> - "Kafka ensures message ordering per chat via partitioning"
> - "Fanout-on-write for groups under 256 users, on-read for larger"
> - "Cassandra for messages - optimized for write-heavy, time-series data"
> - "Three delivery states: sent, delivered, read - each requires an ACK"
